

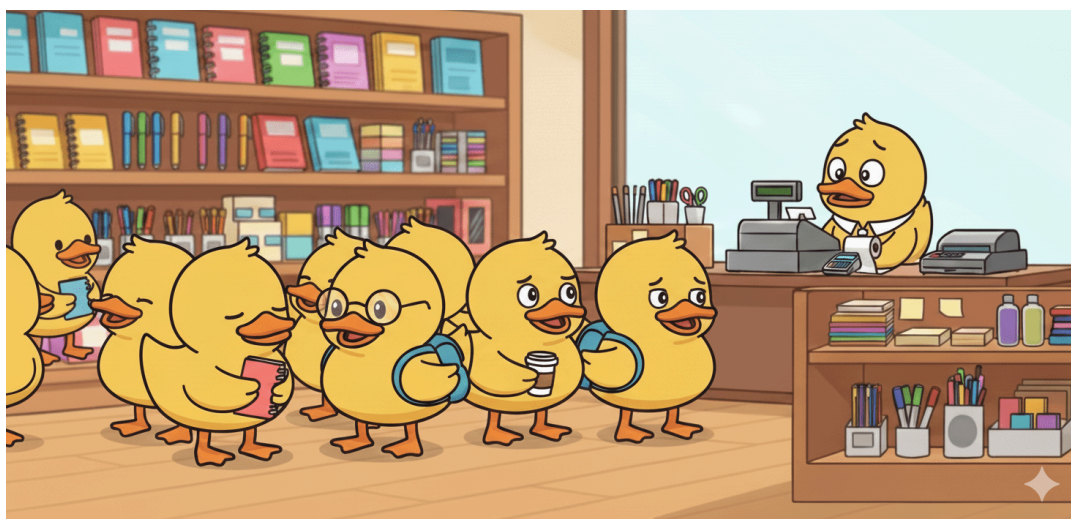


مقدمه

هدف از این تمرین آشنایی شما با `container`-ها و `iterator`-هاست. در این تمرین شما یک نمونه ساده شده از یک برنامه مدیریت فروشگاه را پیاده‌سازی می‌کنید. علاوه بر اجرای درست برنامه، جدا کردن مسئولیت‌ها به تابع‌های مختلف، تمیزی کد، استفاده درست از `container`-ها و `iterator`-ها اهمیت زیادی دارند. سعی کنید در پیاده‌سازی تمرین، توابع مختلفی تعریف کنید که هر کدام تنها یک کار انجام می‌دهند.

زیراکس‌یار

کوروش بعد از اینکه توانست دانش‌آموزان را از کابوس تبدیل واحد نجات دهد، این بار نیز به یاری دانشجویان آمده است. در آغاز هر ترم، زیراکس دانشکده مثل همیشه شلوغ می‌شود؛ دانشجویان پی‌درپی برای چاپ جزوه و خرید لوازم‌التحریر صف می‌کشند. مسئول زیراکس از این حجم سفارش‌ها خسته و درمانده شده و دانشجویان هم نمی‌توانند به‌موقع اقلام مورد نیازشان را تهیه کنند. کوروش تصمیم گرفته برای پایان دادن به این هرج‌ومرج، یک سیستم مدیریت فروشگاه برای زیراکس طراحی و پیاده‌سازی کند.



اضافه کردن کالای جدید

به کمک این دستور یک کالای جدید به فروشگاه اضافه می‌شود. تضمین می‌شود که نام کالاها تکراری نمی‌باشند و شامل کاراکتر فاصله¹ نیز نیستند؛ همچنین کمیت وارد شده عددی صحیح و غیر صفر و مثبت است.

قالب ورودی
add_product <product_name> <quantity>
قالب خروجی
OK

نمونه ورودی
add_product blue_pen 100
نمونه خروجی
OK

افزایش موجودی کالا

به کمک این دستور، موجودی کالایی که از قبل در سیستم موجود است را افزایش می‌دهیم. تضمین می‌شود نام کالای وارد شده در این دستور، حتما در سیستم وجود دارد و کالای جدیدی نیست. همچنین مقدار وارد شده عددی صحیح و غیر صفر و مثبت است.

قالب ورودی
increase_quantity <product_name> <count>
قالب خروجی
OK

¹ Space

نمونه ورودی
increase_quantity blue_pen 50
نمونه خروجی
OK

توضیح مثال

در دستور قبلی ۱۰۰ عدد خودکار آبی به سیستم فروشگاه اضافه شد. به کمک این دستور ۵۰ عدد دیگر خودکار آبی به فروشگاه اضافه شده است. بنابراین موجودی فعلی خودکار آبی برابر ۱۵۰ عدد خواهد بود.

اضافه کردن سفارش جدید

با استفاده از این دستور، یک سفارش جدید ثبت می‌کنیم. هر سفارش دارای اطلاعات زیر است:

- نام سفارش‌دهنده
- نام کالا
- تعداد کالا
- وضعیت سفارش

هر سفارش می‌تواند یکی از این سه وضعیت را داشته باشد:

1. وضعیت Pending: هر سفارش زمانی که ایجاد می‌شود در این وضعیت قرار می‌گیرد تا زمانی که به آن رسیدگی شود.

2. وضعیت Completed: زمانی که به سفارش رسیدگی کرده‌ایم و سفارش با موفقیت به پایان رسیده است.

3. وضعیت Failed: زمانی که به علت کمبود موجودی کالا، سفارش تکمیل نشده است.

بنابراین هر سفارشی که با این دستور ایجاد کرده‌ایم، در وضعیت Pending قرار می‌گیرد. تضمین می‌شود نام کالای وارد شده در این دستور، حتما در سیستم وجود دارد و کالای جدیدی نیست. همچنین تعداد وارد شده عددی صحیح و غیر صفر و مثبت است.

قالب ورودی
add_order <customer_name> <product_name> <quantity>
قالب خروجی
OK

نمونه ورودی
add_order parisa blue_pen 10
نمونه خروجی
OK

رسیدگی به سفارشات

به کمک این دستور، به سفارشات که در وضعیت Pending قرار دارند، رسیدگی می‌کنیم. توجه داشته باشید، سفارشی که زودتر ثبت شده دارای اولویت است و باید زودتر به آن رسیدگی شود. با اجرای این دستور، در صورتی که موجودی کافی از کالای خواسته‌شده وجود داشته باشد، از موجودی آن در سیستم کسر می‌شود و وضعیت سفارش به Completed تغییر می‌کند. در صورتی که موجودی کافی از کالای خواسته‌شده نداشته باشیم، وضعیت سفارش به Failed تغییر می‌کند.

قالب ورودی
process_orders

گزارش‌گیری

برای مدیریت بهتر فروشگاه، انواع مختلفی از گزارش را در سیستم پیاده‌سازی می‌کنیم.

سفارش‌های در حال پردازش

با استفاده از این دستور، سفارشات که در وضعیت Pending قرار دارند را چاپ می‌کنیم. سفارشات را به ترتیب زمانی ایجاد شدن از قدیمی‌ترین تا جدیدترین نمایش دهید. در صورتی که هیچ سفارش در حال پردازشی وجود ندارد، پیام "No pending orders" را چاپ کنید.

قالب ورودی
report_pending
قالب خروجی
1. <customer_name1>: <product_name1> <quantity1> 2. <customer_name2>: <product_name2> <quantity2> ... No pending orders

نمونه ورودی
report_pending
نمونه خروجی
1. parisa: blue_pen 10

تعداد کالای فروخته شده

به کمک این دستور، محاسبه می‌کنیم که از کالای وارد شده چه تعداد فروخته‌ایم. تضمین می‌شود کالای وارد شده در سیستم فروشگاه وجود دارد و کالای جدیدی نیست. توجه داشته باشید، فقط سفارش‌هایی که با موفقیت تکمیل شده‌اند (وضعیت Completed) در محاسبه این تعداد دخیل هستند.

قالب ورودی
sold_quantity <product_name>
قالب خروجی
<sold_qunatity>

نمونه ورودی
sold_quantity blue_pen
نمونه خروجی
10

موجودی فعلی کالاها

به کمک این دستور، موجودی فعلی تمام کالای فروشگاه‌ها را چاپ می‌کنیم. کالاها را به ترتیب الفبایی و صعودی مرتب‌سازی کنید و در خروجی نمایش دهید. در صورتی که هیچ کالایی به سیستم فروشگاه اضافه نشده است، پیام "No products found" چاپ کنید.

قالب ورودی
show_stock
قالب خروجی
1. <product_name1>: <quantity1> 2. <product_name2>: <quantity2> ... No products found

نمونه ورودی
show_stock
نمونه خروجی
1. blue_pen: 140 2. red_pen: 100

نکات و نحوه تحویل

- برنامه شما باید در سیستم عامل لینوکس و با مترجم g++ با استاندارد c++20 یا بالاتر ترجمه و در زمان معقول برای ورودی های آزمون اجرا شود.
- تحویل این تمرین در سامانه گیت‌هاب انجام می‌شود. برای انجام تمرین لطفا از طریق [این لینک](#) وارد شوید. سپس باید شماره دانشجویی خود را انتخاب کنید (دقت کنید که با کمک این شماره دانشجویی به شما نمره خواهیم داد. لطفا در انتخاب درست شماره دانشجویی حتما دقت کنید. در صورتی که به مشکل خوردید با دستیاران ارشد در ارتباط باشید). پس از آن به صفحه‌ای منتقل می‌شوید که در آنجا می‌توانید تمرین جدید را قبول کنید. بعد از قبول کردن تمرین، یک مخزن در [AP Assignments](#) برای شما ساخته می‌شود و باید کدهای خود را در آنجا قرار دهید. برای این تمرین نیاز نیست کار به طور کامل با گیت را بلد باشید و صرفا کد نهایی خود را در فایل src/main.cpp قرار دهید.
- پس از انجام تمرین و بارگذاری در گیت‌هاب، کد Hash آخرین کامیت را به همراه شماره دانشجویی خود در سامانه ای‌لرن آپلود کنید (در خط اول شماره دانشجویی، پس از آن از Enter استفاده کنید و به خط بعد بروید و پس از آن Hash آخرین کامیت). نمونه متن خواسته شده در سامانه ای‌لرن (بخش <last_commit_hash> و <sid> را جایگزین کنید):

```
<sid>
<last_commit_hash>
```

نمونه:

```
810100000
bad8fbcdcf3b9feb371a31e0c370150aa870b18
```

- دقت کنید که عدم رعایت ساختار گفته شده در آپلود یا تغییر ساختار فایل‌ها در مخزن (می‌توانید به دلخواه خود فایل اضافه کنید و ... اما اسم و ساختار فایل‌هایی که در ابتدا به شما داده می‌شود نباید تغییر کند) باعث کسر ۱۰ درصد از نمره شما خواهد شد.
- بخش مهمی از ارزیابی برنامه شما درستی عملکرد آن است. بنابراین به اندازه کافی ورودی‌های آزمایشی طراحی کنید تا درستی خروجی در حالت‌های مختلف آزموده شود و دقت کنید که اگر برنامه شما با مثال‌های داده شده درست کار کند لزوما به معنای درستی در تمام سناریوها نیست.
- درستی برنامه شما از طریق آزمون‌های خودکار سنجیده می‌شود. به این ترتیب، لازم است خروجی تولید شده از نظر بزرگی و کوچکی حروف، رعایت فاصله‌ها، عدم وجود خروجی‌های اضافه، ... دقیقا مانند نمونه‌های داده شده باشد. بنابراین پیشنهاد می‌شود که با استفاده از ابزارهایی مانند diff فرمت خروجی برنامه خود را با خروجی‌هایی که در اختیارتان قرار داده شده است مطابقت دهید.
- هدف این تمرین یادگیری شماست. لطفا تمرین را خودتان انجام دهید. در صورت کشف تقلب مطابق سیاست درس با آن برخورد خواهد شد.

نمرات

- تمیزی کد

- رعایت کردن نام‌گذاری صحیح و منسجم بودن²
- عدم وجود کد تکراری
- رعایت دندانه‌گذاری³
- استفاده صحیح از متغیرهای ثابت⁴ به جای Magic Value-ها⁵
- استفاده صحیح از container-ها و iterator-ها به جای روش‌های قدیمی
- استفاده مناسب از ساختار داده‌های مناسب
- توابع کوتاه که فقط یک کار را انجام می‌دهند

- درستی کد

- آزمون‌های خودکار

دقت کنید که موارد ذکر شده لزوماً کل نمره شما را تشکیل نمی‌دهند و ممکن است با تغییراتی همراه باشند.

² Consistency

³ Indentation

⁴ Constant

⁵ به مقادیر خاصی که در کد استفاده می‌شود و برای عملکرد صحیح کد ضروری است اما دلیل استفاده از آن‌ها مشخص نیست و قابل جایگزین شدن با یک ثابت با اسم مشخص جهت افزایش خوانایی هستند، magic value گفته می‌شود. برای آشنایی بیشتر با این مفهوم می‌توانید [این لینک](#) را مشاهده کنید.